

Module 3

Autoencoders, VAE & Modèles de Diffusion

Reconstruction — espace latent variationnel — génération par débruitage

Du codage compact à la génération

Compresser — structurer le latent — échantillonner des images réalistes

$\hat{x}$

Autoencoder

Reconstruction

$q(z|x)$

VAE

ELBO, KL

ϵ_θ

DDPM

Débruitage

x_{t-1}

DDIM

Sampling rapide

Bassem Ben Hamed

bassem.benhamed@enetcom.usf.tn

ENETCOM — Université de Sfax

Présentation 03 — Module 3

1. Autoencoders : AE, ConvAE & débruitage
2. Variational Autoencoder (VAE)
3. Modèles de diffusion : DDPM, score & DDIM
4. Synthèse & références

Autoencoders : AE, ConvAE & débruitage

Autoencoder — formulation & objectif

Cadre formel

Un autoencoder apprend une **compression non supervisée** : un encodeur $E_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^d$ projette l'entrée vers un *code latent* $z = E_\phi(\mathbf{x})$ ($d \ll D$, *bottleneck*), et un décodeur $D_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^D$ reconstruit $\hat{\mathbf{x}} = D_\theta(z)$. On minimise l'erreur de reconstruction :

$$(\phi^*, \theta^*) = \arg \min_{\phi, \theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\mathcal{L}_{\text{rec}}(\mathbf{x}, D_\theta(E_\phi(\mathbf{x}))) \right].$$

Pertes usuelles : **MSELoss** (pixels continus) ou **BCELoss** (pixels $\in [0, 1]$).

En clair : toute l'image doit passer par un goulot de d nombres avant d'être redessinée — le réseau n'y parvient qu'en retenant l'essentiel (formes, structures), pas les détails accidentels.

Lien avec la PCA

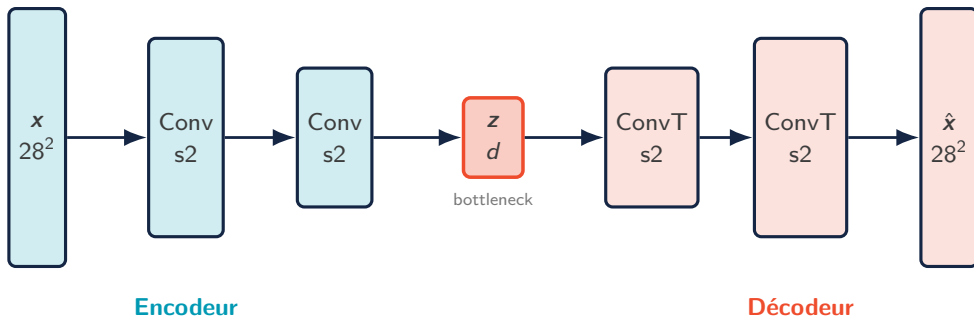
Un AE *linéaire* (E, D sans activation) entraîné en MSE retrouve le **sous-espace des d premières composantes principales**. Les non-linéarités (ReLU, GELU) permettent une variété non linéaire $\rightarrow$ compression plus riche.

Le bottleneck est essentiel

Sans contrainte ($d \geq D$), l'AE apprend l'*identité* (copie triviale). La régularisation vient :

- du **goulot** $d \ll D$ (under-complete)
- ou d'une pénalité (sparse AE, denoising, contractif)

Architecture convolutive — ConvAE



Convolution transposée (upsampling)

Le décodeur restaure la résolution via `nn.ConvTranspose2d`.

Taille de sortie :

$$H_{\text{out}} = (H_{\text{in}} - 1) s - 2p + d(k - 1) + p_{\text{out}} + 1.$$

Exemple : doubler $7 \rightarrow 14$

$H_{\text{in}}=7$, $k=4$, $s=2$, $p=1$, $d=1$, $p_{\text{out}}=0$:

$$(7 - 1) \cdot 2 - 2 + 3 + 0 + 1 = 14.$$

Alternative sans artefacts en damier : `Upsample` + `Conv2d`.

Denoising AE — débruitage par reconstruction

Principe (Vincent et al., 2008)

On corrompt l'entrée par un bruit $x \mapsto \tilde{x} = x + \eta$, puis on demande au réseau de reconstruire l'*original propre* :

$$\mathcal{L}_{\text{DAE}} = \mathbb{E}_{x, \eta} \left\| x - D_{\theta}(E_{\phi}(\tilde{x})) \right\|_2^2.$$

Le réseau ne peut pas copier $\rightarrow$ il apprend la **structure du signal**, pas le bruit.

Intuition : le bruit change à chaque époque ; la seule stratégie gagnante est d'apprendre l'image « propre ».

Types de bruit (imagerie médicale)

- Gaussien $\eta \sim \mathcal{N}(0, \sigma^2 I)$ (IRM)
- Poisson (CT faible dose, comptage de photons)
- Masquage / dropout d'entrée (artefacts)

```
class ConvDAE(nn.Module):
    def __init__(self, d=64):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(3, 32, 3, 2, 1), nn.ReLU(),
            nn.Conv2d(32, d, 3, 2, 1), nn.ReLU())
        self.dec = nn.Sequential(
            nn.ConvTranspose2d(d, 32, 4, 2, 1),
            nn.ReLU(),
            nn.ConvTranspose2d(32, 3, 4, 2, 1),
            nn.Sigmoid())

    def forward(self, x):
        return self.dec(self.enc(x))
```

```
# bruitage a la volée
noisy = (x + 0.2*torch.randn_like(x)).clamp(0,1)
loss = F.mse_loss(model(noisy), x) # cible propre
```

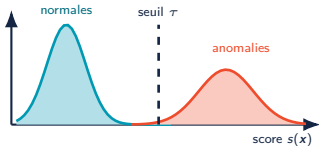
AE — cas d'usage : anomalies, débruitage, exploration latente

Détection d'anomalies non supervisée

On entraîne l'AE *uniquement sur des données normales*. À l'inférence, le score d'anomalie est l'erreur de reconstruction :

$$s(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2, \quad \text{anomalie si } s(\mathbf{x}) > \tau.$$

L'AE n'a jamais vu d'anomalie $\Rightarrow$ il ne sait pas la reconstruire $\Rightarrow s$ élevé. Seuil τ : quantile élevé (p. ex. 99 %) de s sur la validation normale.



Cas d'usage typiques

- **Débruitage** IRM / CT-scan (DAE)
- **Compression** apprise d'images satellites
- **Maintenance prédictive** : capteurs IoT (vibrations, température) — séries temporelles
- **Contrôle qualité** visuel : défauts de fabrication jamais vus à l'entraînement
- **Exploration** : z + UMAP / t-SNE pour visualiser la variété latente

Limite $\rightarrow$ VAE

L'AE déterministe *ne génère pas* : son latent n'a pas de structure probabiliste (zones vides, aucun prior pour échantillonner). Le VAE corrige exactement cela.

Variational Autoencoder (VAE)

VAE — modèle génératif à variable latente

Modèle probabiliste (Kingma & Welling, 2014)

Processus génératif : on tire un code $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, puis une image $\mathbf{x} \sim p_{\theta}(\mathbf{x}|\mathbf{z})$. La vraisemblance d'une image est donc une moyenne sur *tous* les codes :

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (\text{somme sur tous les } \mathbf{z} \text{ possibles}).$$

Problème : cette intégrale est *intractable* (une infinité de $\mathbf{z}$) — et le postérieur $p_{\theta}(\mathbf{z}|\mathbf{x})$ (« quel code a produit cette image ? ») l'est aussi.

Solution : inférence variationnelle

On introduit un **encodeur probabiliste** qui approxime ce postérieur par une gaussienne toute simple :

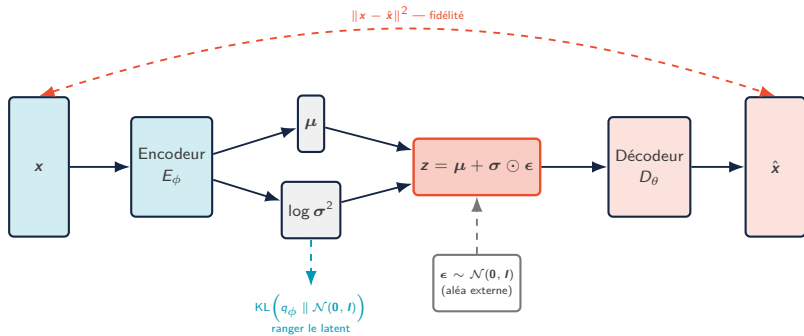
$$q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x})) \quad (\text{indépendante par dimension}).$$

Deux têtes en sortie d'encodeur

Le réseau produit $\mu \in \mathbb{R}^d$ et $\log \sigma^2 \in \mathbb{R}^d$ (on prédit le *log-variance* pour garantir $\sigma^2 > 0$ et stabiliser).

Décodeur $p_{\theta}(\mathbf{x}|\mathbf{z})$: gaussien (MSE) ou Bernoulli (BCE) selon les données.

VAE — architecture : de l'image à la distribution



Lecture du schéma

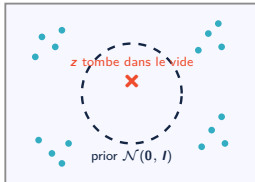
L'encodeur produit *une distribution* $\mathcal{N}(\mu, \sigma^2)$, pas un point : chaque image devient un « nuage » latent, dans lequel on tire z .

Pourquoi deux pertes ?

Reconstruction : nuages informatifs. **KL** : nuages rassemblés autour de l'origine, sans trous — condition pour échantillonner ensuite.

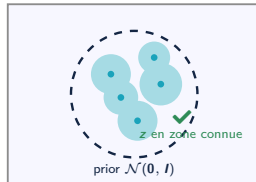
VAE — pourquoi l'AE déterministe ne sait-il pas générer ?

Latent d'un AE : des îlots, des trous



$z \sim \mathcal{N}(\mathbf{0}, I) \rightarrow$ décodeur : **textures incohérentes**

Latent d'un VAE : rangé sous le prior



$z \sim \mathcal{N}(\mathbf{0}, I) \rightarrow$ décodeur : **image plausible**

💡 La contre-épreuve : même tirage z , deux décodeurs

Le décodeur d'un **AE** ne produit que des textures incohérentes : rien n'indique au prior où échantillonner dans ses îlots. Celui d'un **VAE** produit des images plausibles — floues, mais générées *de zéro* : son terme **KL** (flèche cyan du schéma précédent) a aligné le latent sur le prior ($\|\mu\| \ll \sqrt{d}$, sans trous). **AE** = **compression** ; **VAE** = **génération**.

VAE — l'ELBO : la formule à retenir

Ce qu'on maximise à la place de $\log p_\theta(x)$ (incalculable)

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)]}_{\text{reconstruction}} - \underbrace{\text{KL}(q_\phi(z|x) \parallel p(z))}_{\text{régularisation}}$$

Lecture terme à terme

- $z \sim q_\phi(z|x)$: on tire un code dans le **nuage** que l'encodeur associe à x .
- $\log p_\theta(x|z)$: ce code permet-il de bien **re-décoder** x ? (l'objectif de l'AE ; décodeur gaussien $\Rightarrow$ MSE, Bernoulli $\Rightarrow$ BCE).
- $\mathbb{E}[\cdot]$: en moyenne sur les tirages — en pratique, *un seul* tirage par image et par pas suffit.
- $\text{KL}(q_\phi \parallel p)$: distance du nuage au prior $\mathcal{N}(\mathbf{0}, I)$ — le « **loyer** » que paie chaque image pour un latent rangé.
- le signe $-$: bien reconstruire *sans* s'éloigner du prior.

D'où vient la borne ? (en une phrase)

On montre que $\log p_\theta(x) = \mathcal{L}_{\text{ELBO}} + \text{une KL} \geq \mathcal{L}_{\text{ELBO}}$: l'ELBO est une **borne inférieure** calculable de la vraisemblance. La maximiser pousse $\log p_\theta(x)$ vers le haut — sans jamais calculer l'intégrale intractable.

En clair

Reconstruction = nuages **informatifs** ; KL = nuages **rassemblés, sans trous**. Les deux ensemble = un latent où l'on pourra *échantillonner*.

VAE — reparametrization trick : faire passer le gradient

Le problème

Échantillonner $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$ est une opération *aléatoire* : le gradient ne traverse pas un tirage. Impossible d'entraîner l'encodeur à travers un `torch.normal`.

L'astuce : sortir l'aléa du chemin

$$\mathbf{z} = \mu(\mathbf{x}) + \sigma(\mathbf{x}) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

μ, σ : fonctions *déterministes* de $\mathbf{x}$, le gradient y passe ; ϵ : simple **entrée externe** tirée hors du réseau (la flèche pointillée du schéma). Même loi pour $\mathbf{z}$, mais différentiable.

Vérification à la main

$\mathbf{z} = \mu + \sigma \epsilon$ est une fonction *ordinaire* de μ et σ :

$$\frac{\partial \mathbf{z}}{\partial \mu} = 1 \qquad \frac{\partial \mathbf{z}}{\partial \sigma} = \epsilon$$

Un `backward` le confirme : `mu.grad = 1`, `std.grad =  $\epsilon$` .

En PyTorch

`dist.sample()` → `grad_fn = None` : graphe **coupé**.
`dist.rsample()` ou `mu + std*randn_like(std)` → le gradient **circule**.

VAE — la KL en forme close : terme à terme & à la main

Entre le nuage $q = \mathcal{N}(\mu, \sigma^2)$ et le prior $p = \mathcal{N}(0, I)$

$$\text{KL}(q \parallel p) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1) \quad (\text{exacte, sans échantillonnage — une ligne de code})$$

Lecture terme à terme (dimension par dimension)

- μ_j^2 : pénalise un nuage *décentré* $\Rightarrow$ pousse la moyenne **vers 0**.
- $\sigma_j^2 - \log \sigma_j^2 - 1$: minimal en $\sigma_j=1 \Rightarrow$ pousse la variance **vers 1** (ni nuage écrasé, ni nuage géant).
- $\sum_j$: chaque dimension latente paie **son propre loyer**.
- Total ≥ 0 , nul ssi $\mu_j=0$, $\sigma_j=1$ partout : le nuage = le prior exact.

Exemple à la main ($d=2$)

$\mu = (0.5, -0.3)$, $\sigma = (0.8, 1.2)$:

$$j=1 : 0.25 + 0.64 - \log 0.64 - 1 = 0.336$$

$$j=2 : 0.09 + 1.44 - \log 1.44 - 1 = 0.165$$

$$\text{KL} = \frac{1}{2}(0.501) = \mathbf{0.250} \text{ nats.}$$

Dans le code

```
kl = -0.5*sum(1 + lv - mu**2 - exp(lv)) avec lv  
= log  $\sigma^2$  — ce que renvoie aussi kl_divergence(q, p).
```

VAE — fonction de perte & implémentation

Perte à minimiser ($-\mathcal{L}_{\text{ELBO}}$)

$$\mathcal{L}_{\text{VAE}} = \underbrace{\|x - \hat{x}\|_2^2}_{\text{recon (MSE)}} + \beta \cdot \underbrace{\text{KL}(q_\phi \| p)}_{\text{forme close}}.$$

β -VAE (Higgins et al., 2017) : $\beta > 1$ force un latent désenchevêtré (disentangled), chaque dimension capturant un facteur de variation indépendant.

KL annealing

Démarrer $\beta_{\text{eff}} \approx 0$ puis monter linéairement sur les premières epochs (`beta*min(1, ep/warmup)`) évite le **posterior collapse** ($q_\phi \rightarrow p$, latent ignoré).

```
class VAE(nn.Module):
    def __init__(self, D, d):
        super().__init__()
        self.fc = nn.Linear(D, 400)
        self.mu = nn.Linear(400, d)
        self.lv = nn.Linear(400, d) # log var
        self.dec = nn.Sequential(
            nn.Linear(d, 400), nn.ReLU(),
            nn.Linear(400, D), nn.Sigmoid())

    def forward(self, x):
        h = F.relu(self.fc(x))
        mu, lv = self.mu(h), self.lv(h)
        std = torch.exp(0.5*lv) # reparam
        z = mu + std*torch.randn_like(std)
        return self.dec(z), mu, lv

    def loss_fn(xh, x, mu, lv, beta=1.0):
        rec = F.mse_loss(xh, x, reduction='sum')
        kl = -0.5*torch.sum(1+lv-mu.pow(2)-lv.exp())
        return rec + beta*kl
```

VAE — calcul à la main : tout le schéma en chiffres

Déroulé du forward (mini-VAE : $D=4$ pixels, latent $d=2$, $\beta=1$)

1. Entrée : $\mathbf{x} = (0.9, 0.1, 0.8, 0.2)$.

2. Encodeur E_ϕ — les deux têtes sortent :

$$\boldsymbol{\mu} = (0.5, -0.3) \quad \log \boldsymbol{\sigma}^2 = (-0.45, 0.36)$$

d'où $\boldsymbol{\sigma} = e^{\frac{1}{2} \log \boldsymbol{\sigma}^2} = (0.80, 1.20)$ (std = $\exp(0.5 \cdot \log \boldsymbol{\sigma}^2)$)

3. Tirage $\boldsymbol{\epsilon} = (1.0, -0.5)$, puis reparamétrisation :

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon} = (0.5 + 0.8 \cdot 1.0, -0.3 + 1.2 \cdot (-0.5)) = (1.3, -0.9)$$

4. Décodeur D_θ : $\hat{\mathbf{x}} = D_\theta(\mathbf{z}) = (0.8, 0.2, 0.7, 0.2)$.

Les deux termes de l'ELBO, en chiffres

Reconstruction (flèche orange) :

$$\|\mathbf{x} - \hat{\mathbf{x}}\|^2 = 0.1^2 + 0.1^2 + 0.1^2 + 0^2 = 0.03$$

KL (flèche cyan ; mêmes $\boldsymbol{\mu}, \boldsymbol{\sigma}$ que l'exemple KL) :

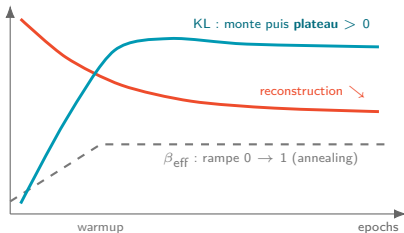
$$\text{KL} = \frac{1}{2} (0.336 + 0.165) = 0.250$$

$$\mathcal{L}_{\text{VAE}} = -\mathcal{L}_{\text{ELBO}} = 0.03 + 1 \cdot 0.250 = \mathbf{0.28}$$

À remarquer

- $\boldsymbol{\epsilon} = (0, 0)$ donnerait $\mathbf{z} = \boldsymbol{\mu}$: chaque tirage $\Rightarrow$ un $\mathbf{z}$ différent, la même image produit un *nuage*.
- Ici $\text{KL} \gg \text{recon}$: la perte tire $\boldsymbol{\mu} \rightarrow \mathbf{0}$, $\boldsymbol{\sigma} \rightarrow \mathbf{1}$.
- Le gradient traverse $\boldsymbol{\mu}, \boldsymbol{\sigma}$; $\boldsymbol{\epsilon}$ n'est qu'une entrée (le trick).

VAE — à l'entraînement : lire les courbes rec / KL



La KL monte — l'encodeur code de l'information — puis se stabilise au-dessus de zéro : le latent est utilisé et rangé.

Ce qu'on veut voir

- **rec** : baisse régulière — le VAE apprend à reconstruire ;
- **KL** : se stabilise à une valeur **strictement positive** = le latent transporte de l'information.

Drapeau rouge : KL $\rightarrow 0$ (posterior collapse)

q_ϕ s'écrase sur le prior : latent **ignoré**, toutes les générations se ressemblent. Remède : le **KL annealing** — laisser l'encodeur apprendre à coder *avant* de le ranger.

β -VAE — le curseur fidélité $\leftrightarrow$ structure du latent

La KL se décompose dimension par dimension

$KL = \sum_j KL_j$ avec $KL_j = \frac{1}{2} (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$ moyennée sur le dataset. $KL_j \approx 0 \Rightarrow$ dimension **morte** : collée au prior, elle ne transporte *aucune* information.

Exemple mesuré (ConvVAE, DermaMNIST, $d=128$)

β	val rec	val KL	dims actives
0,1	25,7	96,0	128 / 128
1	36,0	21,6	35 / 128
4	66,5	9,3	16 / 128

$\beta \uparrow$: KL et dimensions actives $\downarrow$ (latent parcimonieux), mais reconstruction *plus floue* — le compromis **rate-distortion**.

Comment régler β ?

- $\beta \ll 1$: KL débranchée $\rightarrow$ on retombe vers l'**AE** (recon nette, génération dégradée).
- $\beta = 1$: l'ELBO standard.
- $\beta > 1$: chaque dimension paie un loyer plus cher $\rightarrow$ l'info se **concentre** sur quelques facteurs de variation = latent **désenchevêtré** (Higgins et al., 2017).

Traversée latente : visualiser les facteurs

Sur le modèle $\beta=4$: faire varier une dimension active $z_j \in [-3, 3]$ et décoder — chaque dimension déplace un facteur interprétable (teinte, taille, contraste).

VAE — génération, conditionnement & interpolation

Génération

Après entraînement, on *jette l'encodeur* : on échantillonne $z \sim \mathcal{N}(\mathbf{0}, I)$ et $\hat{x} = D_\theta(z)$. Le terme KL garantit que le prior couvre bien la région utile du latent.

Conditional VAE (CVAE)

On concatène un label y (one-hot ou `nn.Embedding`) à l'entrée encodeur et au latent décodeur :

$$q_\phi(z|x, y), \quad p_\theta(x|z, y).$$

⇒ génération **contrôlée par classe**.

Interpolation latente

Entre deux codes z_a, z_b , on décode des points intermédiaires :

$$(\text{lerp}) \quad z_t = (1-t) z_a + t z_b, \quad t \in [0, 1].$$

Mieux : le **slerp** (interpolation *sphérique*) suit l'*arc* plutôt que la corde → il garde la norme typique $\sqrt{d}$ du prior gaussien, d'où un *morphing* plus réaliste (évite de traverser des z improbables près de $\mathbf{0}$).

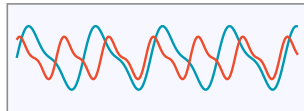
À retenir

Le décodeur seul est le **générateur** ; l'encodeur ne sert qu'à l'entraînement (et à projeter de nouvelles données dans le latent).

VAE 1D — générer des séries temporelles capteurs

Du 2D au 1D : ce qui change... et ce qui ne change pas

- **Rien** côté maths : encodeur $\rightarrow (\mu, \log \sigma^2) \rightarrow \text{reparam} \rightarrow$ décodeur ; perte rec + KL, identiques.
- `nn.Conv2d` $\rightarrow$ `nn.Conv1d` (et `ConvTranspose1d`) : on convolue le *temps* au lieu de l'espace.
- **Pas de sigmoïde** en sortie : un signal n'est pas borné dans $[0, 1]$ (décodeur gaussien sur $\mathbb{R}$, MSE inchangée).
- Latent minuscule ($d=8$ suffit) : un signal $\approx$ quelques facteurs (fréquence, amplitude, phase...).



deux tirages $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ décodés

Exemple : vibrations machine

Signaux de vibration (fondamentale + harmonique 2 + bruit).
Le décodeur produit des trajectoires **réalistes jamais vues** :
augmentation de données pour la **maintenance prédictive** —
les pannes sont rares, donc chères à collecter.

VAE — cas d'usage

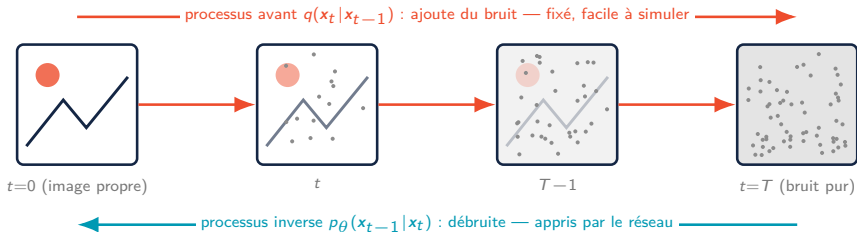
Ce qu'on exploite selon l'application

Cas d'usage	Brique exploitée	Exemple concret
Augmentation de données	décodeur + CVAE (y)	images synthétiques pour classes rares (lésions dermatologiques minoritaires)
Détection d'anomalies / OOD	score ELBO (ou recon.)	ELBO faible = échantillon improbable ; plus calibré que l'AE déterministe
Séries temporelles synthétiques	décodeur (latent 1D)	trajectoires capteurs réalistes pour tester une chaîne de maintenance prédictive
Imputation / débruitage	encodeur $\rightarrow$ décodeur	compléter des mesures manquantes en passant par le latent
Édition sémantique	arithmétique latente	$z + \lambda v_{\text{attribut}}$: déplacer un facteur de variation (β -VAE)
Compression du calcul	latent compact	base des Latent Diffusion Models (section suivante)

Limite connue : reconstructions un peu **floues** (moyenne sous la gaussienne du décodeur) — motivation directe des modèles de diffusion.

Modèles de diffusion : DDPM, score & DDIM

Diffusion — intuition & processus avant



Analogie : une goutte d'encre qui se dilue — l'avant est trivial à simuler, le réseau apprend à « rembobiner le film ».

Processus avant (forward) — fixé, sans aucun paramètre appris

Chaîne de Markov : à chaque pas on *atténue* un peu l'image et on *ajoute* un peu de bruit gaussien (planning $\{\beta_t\}_{t=1}^T$) :

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t; \underbrace{\sqrt{1 - \beta_t} \mathbf{x}_{t-1}}_{\text{image atténuée}}, \underbrace{\beta_t \mathbf{I}}_{\text{bruit ajouté}}\right).$$

β_t petit = petit pas. Après T pas, $\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$: toute l'information de $\mathbf{x}_0$ a disparu. Rien n'est appris ici — ce processus est **fixé d'avance**.

Diffusion — propriété clé : saut direct vers x_t

Marginale en forme close — le « saut direct »

Posons $\alpha_t = 1 - \beta_t$ et $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ (la *fraction de signal* encore présente à t). La composition des gaussiennes donne, *sans itérer* :

$$\mathbf{x}_t = \underbrace{\sqrt{\bar{\alpha}_t} \mathbf{x}_0}_{\text{signal restant}} + \underbrace{\sqrt{1 - \bar{\alpha}_t} \epsilon}_{\text{bruit}}, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, I).$$

$\bar{\alpha}_t$ va de 1 (image nette) à 0 (bruit pur). $\Rightarrow$ on fabrique $\mathbf{x}_t$ à **n'importe quel pas t en une seule opération** — c'est ce qui rend l'entraînement efficace.

Exemple numérique (pixel scalaire)

Soit $\bar{\alpha}_t = 0.64 \Rightarrow \sqrt{\bar{\alpha}_t} = 0.8$, $\sqrt{1 - \bar{\alpha}_t} = 0.6$. Avec $x_0 = 1.0$ et $\epsilon = 0.5$:

$$x_t = 0.8 \cdot 1.0 + 0.6 \cdot 0.5 = 1.1.$$

Rapport signal/bruit : $\text{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t} = \frac{0.64}{0.36} \approx 1.78$.

Fin de chaîne : $\bar{\alpha}_T \approx 4 \cdot 10^{-5} \Rightarrow x_T \approx \epsilon$ (moyenne ≈ 0 , écart-type ≈ 1) : du **bruit pur**, comme promis.

Le pas inverse *idéal* (si on connaissait x_0)

Connaissant x_0 , revenir de x_t à x_{t-1} est gaussien et *calculable* : $q(x_{t-1} | x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$. C'est la **cible** que le réseau devra imiter *sans* connaître x_0 .

DDPM — processus inverse & objectif variationnel

Processus inverse paramétré (Ho et al., 2020)

Un réseau apprend à inverser le bruitage : $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$, en partant de $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Borne variationnelle — mêmes outils que le VAE

Comme pour le VAE, on majore $\mathbb{E}[-\log p_\theta(\mathbf{x}_0)]$ par une somme de termes qui se ramènent, à chaque pas t , à une **KL entre deux gaussiennes** :

$$L_{t-1} = \text{KL} \left(\underbrace{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}_{\text{pas inverse idéal}} \parallel \underbrace{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}_{\text{pas inverse appris}} \right).$$

Lecture : apprendre = rendre le pas inverse *appris* aussi proche que possible du pas inverse *idéal*, à chaque t . (Les termes de bord ne s'apprennent pas.)

Que doit prédire le réseau ?

Le saut direct se **renverse** : qui connaît le bruit connaît l'image, $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon) / \sqrt{\bar{\alpha}_t}$. La moyenne optimale du pas inverse se réécrit donc entièrement en fonction du bruit : il suffit que le réseau **prédise ϵ** , pas l'image. (La formule exacte réapparaît telle quelle dans le pas d'échantillonnage.)

À retenir : toute cette machinerie variationnelle se réduit en pratique à entraîner un réseau qui **devine le bruit ajouté** — la perte concrète arrive à la slide suivante.

DDPM — perte simplifiée & entraînement

Objectif simplifié L_{simple} — la formule à retenir

En laissant tomber la pondération issue de la KL (cela *améliore* même la qualité), tout l'entraînement se réduit à :

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(x_t, t)\|_2^2 \right]$$

- $\mathbb{E}_{t, x_0, \epsilon}$: une image, un pas t au hasard (tous les niveaux de bruit comptent), un bruit tiré ;
- ϵ vs $\epsilon_{\theta}(x_t, t)$: bruit *ajouté* vs bruit *deviné* $\Rightarrow$ **un simple MSE**, stable (aucun adversaire ; $\text{loss} \approx 0,5 \rightarrow 0,02$).

Algorithme 1 — Training

1. $x_0 \sim \mathcal{D}$
2. $t \sim \mathcal{U}\{1, \dots, T\}$
3. $\epsilon \sim \mathcal{N}(0, I)$
4. $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
5. Pas de gradient sur $\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$

```
betas = torch.linspace(1e-4, 0.02, T)
abar = torch.cumprod(1.0 - betas, dim=0)
def train_step(model, x0):
    t = torch.randint(0, T, (x0.size(0),))
    eps = torch.randn_like(x0)
    a = abar[t].sqrt().view(-1,1,1,1)
    b = (1 - abar[t]).sqrt().view(-1,1,1,1)
    x_t = a*x0 + b*eps          # saut direct
    eps_hat = model(x_t, t)     # U-Net
    return F.mse_loss(eps_hat, eps)
```

DDPM — calcul à la main : bruiteur, deviner, débruiter

Un pixel, les chiffres du saut direct : $x_0 = 1.0$, $\bar{\alpha}_t = 0.64$ ($\sqrt{\bar{\alpha}_t} = 0.8$, $\sqrt{1 - \bar{\alpha}_t} = 0.6$), bruit tiré $\epsilon = 0.5$.

Les 3 étapes en chiffres

1. Bruiter (saut direct, fixé) :

$$x_t = 0.8 \cdot 1.0 + 0.6 \cdot 0.5 = 1.1$$

2. Deviner (appris) — le réseau voit ($x_t=1.1$, t) et prédit $\epsilon_\theta = 0.4$:

$$L_{\text{simple}} = (\epsilon - \epsilon_\theta)^2 = (0.5 - 0.4)^2 = 0.01$$

3. Débruiter (algèbre) — on renverse le saut direct avec le bruit *deviné* :

$$\hat{x}_0 = \frac{x_t - 0.6 \epsilon_\theta}{0.8} = \frac{1.1 - 0.24}{0.8} = 1.075$$

Avec une prédiction **parfaite** ($\epsilon_\theta=0.5$) : $\hat{x}_0 = \frac{1.1-0.3}{0.8} = \mathbf{1.0}$ — on retrouve x_0 *exactement*.

Comment interpréter ?

- Deviner le bruit $\Leftrightarrow$ savoir remonter à l'image : les deux portent la **même information**.
- L'erreur 0.1 sur ϵ devient $\frac{0.6}{0.8} \cdot 0.1 = 0.075$ sur $\hat{x}_0$; le facteur $\sqrt{1 - \bar{\alpha}_t} / \sqrt{\bar{\alpha}_t}$ **explose** quand $\bar{\alpha}_t \rightarrow 0$ (t grand).
- D'où la stratégie **itérative** : ne retirer qu'une *partie* du bruit à chaque pas (slide suivante), plutôt que sauter à $\hat{x}_0$ d'un coup.

À retenir

Tout le modèle tient dans ce va-et-vient : **bruiter** (fixé), **deviner** (appris), **débruiter** (algèbre).

DDPM — échantillonnage (sampling)

Pas de débruitage ancestral

Partant de $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, pour $t = T, \dots, 1$:

$$\mathbf{x}_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)}_{\text{on retire le bruit prédit}} + \underbrace{\sigma_t \mathbf{z}}_{\text{petit bruit réinjecté}} .$$

On *enlève* une part du bruit deviné ϵ_{θ} , puis on *rajoute* un petit aléa $\sigma_t \mathbf{z}$ ($\mathbf{z}=\mathbf{0}$ au dernier pas). Répété T fois $\rightarrow$ image.

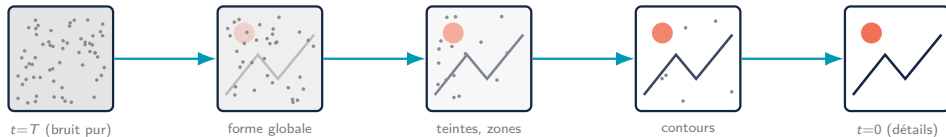
Coût

DDPM exige T **passes réseau** (typiquement $T=1000$) $\rightarrow$ génération lente. D'où DDIM (plus loin).

```
@torch.no_grad()
def sample(model, shape, T):
    x = torch.randn(shape)           #  $x_T$ 
    for t in reversed(range(T)):
        z = torch.randn_like(x) if t>0 else 0
        eps = model(x, torch.full((shape[0],), t))
        a, ab, b = alphas[t], abar[t], betas[t]
        mean = (x - b / (1-ab).sqrt() * eps) / a.sqrt()
        x = mean + b.sqrt() * z       #  $\sigma_t = \sqrt{\beta_t}$ 
    return x                          #  $x_0$  genere
```

Chaque itération = 1 forward du U-Net + une injection de bruit (sauf au dernier pas).

DDPM — la génération construit l'image du grossier au fin



Ce qu'on observe le long de la trajectoire

En suivant x_t pour t décroissant : les **basses fréquences** (forme, teinte globale) émergent du bruit en premier ; les **détails** (contours, textures) n'apparaissent qu'aux petits t . Chaque pas $p_\theta(x_{t-1}|x_t)$ rapproche l'échantillon de la variété des images réelles.

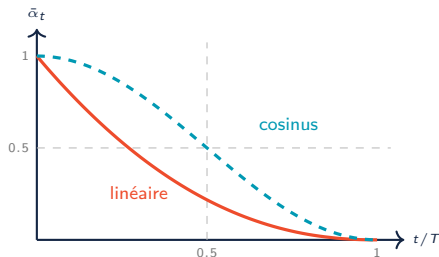
Pourquoi c'est important

Ce raffinement **itératif** récupère les hautes fréquences que le VAE laissait **floues** (une seule passe de décodage) — la raison du bond en qualité des modèles de diffusion.

Noise schedule — linéaire vs cosinus

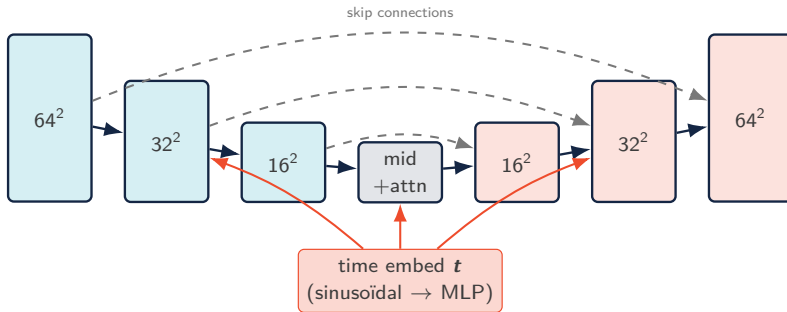
Choix du schedule $\{\beta_t\}$

- **Linéaire** (Ho 2020) : β_t croît linéairement de 10^{-4} à 0.02 ($T=1000$). Simple, mais détruit le signal trop vite en fin de chaîne.
- **Cosinus** (Nichol & Dhariwal, 2021) : on impose à $\bar{\alpha}_t$ une descente en *cosinus carré* (au lieu de la laisser découler d'un β_t linéaire). Le signal décroît plus *douce*ment → on en garde plus longtemps → meilleure qualité.



Le schedule cosinus garde plus de signal au milieu de la chaîne.

Architecture U-Net pour la diffusion



Composants standards

- Encodeur-décodeur conv. avec **skip connections**
- **GroupNorm + SiLU** (Swish) au lieu de BatchNorm
- Blocs résiduels à chaque résolution
- **Self-attention** aux résolutions basses (16^2 , 8^2)

Time embedding

Le pas t est encodé en **positional encoding sinusoïdal**, projeté par un MLP, puis injecté dans chaque bloc résiduel (addition ou FiLM). Un *seul* réseau $\epsilon_{\theta}(\cdot, t)$ sert pour tous les t .

Vue score-based — l'autre lecture de la diffusion

Score matching (Song & Ermon, 2019)

Le **score** d'une densité est $s(x) = \nabla_x \log p(x)$: une *boussole* qui pointe vers les zones de forte densité (« par où rendre l'image plus plausible »). Sur l'image bruitée, score et bruit prédit sont *proportionnels* :

$$s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}.$$

⇒ **prédire le bruit** $\equiv$ **estimer le score** (au signe et à l'échelle près). Débruiter, c'est *suivre la boussole* vers les images plausibles.

Échantillonnage par Langevin

Connaissant le score, on remonte la densité :

$$x \leftarrow x + \frac{\delta}{2} s_\theta(x) + \sqrt{\delta} z.$$

DDPM et score-based sont **deux faces du même cadre** (SDE, Song et al. 2021).

Pourquoi c'est important

- Unifie diffusion discrète (DDPM) et continue (SDE)
- Justifie les solveurs ODE rapides (DPM-Solver)
- Le même ϵ_θ sert dans les deux formalismes

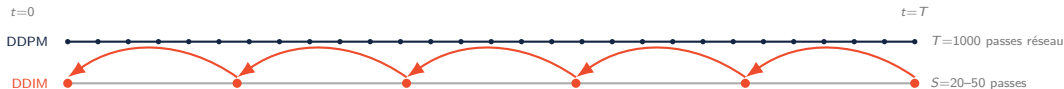
DDIM — échantillonnage accéléré & déterministe

Song, Meng & Ermon (2021) — processus non markovien

DDIM réutilise *le même réseau* ϵ_θ mais redéfinit le pas inverse. On prédit d'abord x_0 , puis on réinjecte :

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}, \quad x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta + \sigma_t z.$$

Étape 1 : estimer l'image finale $\hat{x}_0$ (retirer le bruit prédit de x_t). **Étape 2** : re-bruiter $\hat{x}_0$ au niveau $t-1$. $\sigma_t=0 \Rightarrow$ trajectoire **déterministe** $\rightarrow$ *grands sauts*.



Cas déterministe $\sigma_t = 0$

Le terme stochastique disparaît : la trajectoire $x_T \rightarrow x_0$ devient une **ODE déterministe**. Même $x_T \Rightarrow$ même image (latent reproductible, interpolation possible).

Accélération $\times 10$ à $\times 50$

On échantillonne sur une **sous-suite** de $S \ll T$ pas ($S=20-50$) *sans réentraîner*. DDPM ($\sigma_t = \sqrt{\bar{\beta}_t}$) et DDIM ($\sigma_t=0$) : deux extrêmes d'une famille paramétrée par σ_t (le **eta** des implémentations).

Diffusion conditionnelle & Classifier-Free Guidance

Conditionnement

On rend $\epsilon_\theta(\mathbf{x}_t, t, \mathbf{c})$ dépendant d'une condition $\mathbf{c}$:

- *class-conditional* : embedding de label ajouté au time embedding
- *text-conditional* : embeddings CLIP via **cross-attention**
- *image-conditional* : concaténation au canal d'entrée (inpainting, SR)

Classifier-Free Guidance (Ho & Salimans, 2021)

On entraîne *un seul* réseau conditionnel et inconditionnel (en dropant $\mathbf{c} = \emptyset$ aléatoirement).

Formule de guidage

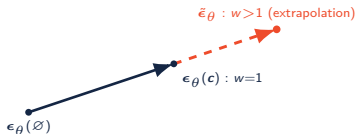
À l'échantillonnage, on *extrapole* entre prédiction conditionnelle et inconditionnelle :

$$\tilde{\epsilon}_\theta = \epsilon_\theta(\emptyset) + w(\epsilon_\theta(\mathbf{c}) - \epsilon_\theta(\emptyset)).$$

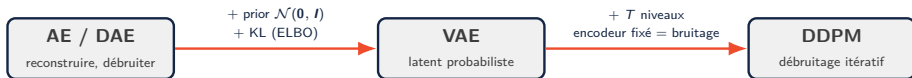
$\epsilon_\theta(\emptyset)$: prédiction *sans* condition ; la différence $\epsilon_\theta(\mathbf{c}) - \epsilon_\theta(\emptyset)$ pointe vers la condition ; w règle de combien on pousse.

- $w = 0$: inconditionnel ; $w = 1$: conditionnel standard
- $w > 1$: **amplifie** l'adhérence à $\mathbf{c}$ (qualité $\uparrow$, diversité $\downarrow$)

Typiquement $w \in [3, 8]$ pour le texte $\rightarrow$ image.



Situer la diffusion : AE $\rightarrow$ VAE $\rightarrow$ DDPM



Tout le module converge ici

- Le débruiteur ϵ_θ est un **U-Net**, et sa tâche est celle du **denoising AE** — simplement conditionnée par t : un seul réseau pour tous les niveaux de bruit.
- La **borne variationnelle**, la KL entre gaussiennes et la reparamétrisation viennent du **VAE**.
- Un DDPM est un **VAE hiérarchique** à T niveaux dont l'encodeur est **fixé** (le bruitage) : tout l'effort d'apprentissage porte sur le décodage progressif.

Ce que la diffusion corrige

Le **fou** du VAE (une seule passe de décodage) disparaît : T petits pas de débruitage reconstruisent les **hautes fréquences** une à une.

Le prix

L'échantillonnage coûte S passes réseau (contre 1 pour AE, VAE, GAN) — d'où DDIM et les solveurs rapides.

Diffusion — cas d'usage & comparaison des modèles génératifs

Cas d'usage & variantes

- **Texte → image** : Latent Diffusion (Stable Diffusion) — diffusion dans le latent d'un VAE, coût divisé
- **Inpainting** : compléter une zone masquée (restauration, anonymisation)
- **Super-résolution** : conditionnée basse résolution (SR3, Palette)
- **Augmentation de données** : images synthétiques pour classes rares
- **Séries temporelles** : génération / imputation de signaux capteurs (IoT)

Déclinaisons médicales

Synthèse IRM/CT pour équilibrer un jeu de données, super-résolution CT basse dose, restauration d'images cliniques artefactées.

Critère	VAE	GAN	Diffusion
Qualité	Moyenne (flou)	Élevée	Très élevée
Diversité	Bonne	Faible (collapse)	Excellente
Stabilité	Stable	Instable	Stable
Vitesse éch.	1 passe	1 passe	Lente (S pas)
Latent	Explicite	Implicite	Implicite

GAN & ViT seront traités au Module 4 ; diffusion = état de l'art en qualité/diversité.

Prototypage avec diffusers (Hugging Face)

Briques de la bibliothèque

- `UNet2DModel` : U-Net conditionné par timestep prêt à l'emploi
- `DDPMScheduler` / `DDIMScheduler` : gèrent $\beta_t, \alpha_t, \bar{\alpha}_t$ et les pas inverses
- `DDPMPipeline` : boucle d'échantillonnage clé en main

Bonne pratique

Entraîner avec `DDPMScheduler` ($T=1000$) puis échantillonner avec `DDIMScheduler` ($S=50$) : même modèle, génération $\times 20$ plus rapide.

```
from diffusers import UNet2DModel, DDPMScheduler

model = UNet2DModel(
    sample_size=64, in_channels=3,
    out_channels=3,
    block_out_channels=(64,128,256,256),
    down_block_types=("DownBlock2D",)*2
    + ("AttnDownBlock2D", "DownBlock2D"))

sched = DDPMScheduler(num_train_timesteps=1000)

# --- une etape d'entrainement ---
noise = torch.randn_like(x0)
t = torch.randint(0, 1000, (x0.size(0),))
x_t = sched.add_noise(x0, noise, t) # saut direct
pred = model(x_t, t).sample        # eps_theta
loss = F.mse_loss(pred, noise)
```

Synthèse & références

Synthèse mathématique du Module 3

Concept	Formule clé	PyTorch / outil
AE reconstruction	$\min_{\phi, \theta} \ x - D_{\theta}(E_{\phi}(x))\ _2^2$	<code>nn.Conv2d</code> / <code>ConvTranspose2d</code>
Denoising AE	cible propre depuis entrée bruitée $\tilde{x}$	<code>x + randn_like(x)</code>
VAE — ELBO	$\mathbb{E}_q[\log p_{\theta}(x z)] - \text{KL}(q_{\phi}\ p)$	deux têtes <code>mu</code> , <code>logvar</code>
Reparam. trick	$z = \mu + \sigma \odot \epsilon, \epsilon \sim \mathcal{N}(0, I)$	<code>mu + std*randn_like</code>
KL gaussienne	$-\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$	forme close (1 ligne)
β -VAE	$\mathcal{L}_{\text{rec}} + \beta \text{KL (disentanglement)}$	<code>beta</code> hyperparamètre
Forward diffusion	$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$	<code>torch.cumprod(alphas)</code>
DDPM loss	$\mathbb{E}_{t, x_0, \epsilon} \ \epsilon - \epsilon_{\theta}(x_t, t)\ ^2$	<code>F.mse_loss(eps_hat, eps)</code>
DDPM sampling	$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} (x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}) + \sigma_t z$	boucle <code>reversed(range(T))</code>
Score $\leftrightarrow$ bruit	$s_{\theta} = -\epsilon_{\theta} / \sqrt{1 - \bar{\alpha}_t}$	<code>DDPMScheduler</code>
DDIM ($\sigma=0$)	déterministe, sous-suite $S \ll T$	<code>DDIMScheduler</code>
Classifier-free guid.	$\epsilon_{\emptyset} + w(\epsilon_c - \epsilon_{\emptyset})$	<code>guidance_scale=w</code>

Trois ouvrages couvrent les concepts présentés

1. **Atienza, R.** (2018). *Advanced Deep Learning with Keras*. Packt Publishing. ISBN 978-1-78862-941-6.

Chapitres pertinents : autoencoders, VAE, modèles génératifs profonds.

2. **Vasilev, I.** (2019). *Python Deep Learning* (2^e éd.). Packt Publishing.

Chapitres pertinents : autoencoders, modèles génératifs, espace latent.

3. **Buduma, N., Buduma, N., & Papa, J.** (2022). *Fundamentals of Deep Learning* (2^e éd.). O'Reilly Media. ISBN 978-1-492-08128-7.

Chapitres pertinents : modèles génératifs, inférence variationnelle, débruitage.

Les références couvrent les fondements théoriques de toutes les architectures vues dans la formation (Modules 1 à 4).

Compresser, structurer, générer.

Cap sur le Module 4 — GAN, Vision Transformers & déploiement

✉ bassem.benhamed@enetcom.usf.tn